

Staff Software Engineer (Backend) - Assignment

Introduction

At BUX, we are building backend software that is scalable, low-latency, and event-driven. Our app-facing services are responsible for aggregating data from multiple domains and delivering a seamless, real-time experience to our mobile applications.

This assignment offers you the opportunity to showcase your expertise in designing architectures that integrate multiple data sources, handle consistency trade-offs, and deliver low-latency, app-facing APIs.

Objective

Create a **production-ready architecture document** for an Investment Plans feature that allows users to create and manage recurring investment plans through the BUX mobile app.

Estimated Time: 4–6 hours (quality is valued over speed)

Deliverable: Architecture document (code samples optional)

Important Note

This is a substantial assignment, and we recognise you may have other commitments. Please focus on demonstrating your architectural thinking and judgment rather than providing exhaustive documentation.

We are seeking Staff-level insight, not perfection.

Product Context

Many users want to invest regularly but forget or don't have time to manually place orders. Investment Plans let users "set and forget" – automating their recurring investments.

User goals:

- Invest a fixed amount regularly without manual effort
- Diversify across multiple instruments in one plan
- Stay informed about what happened with their money

Business goals:

- Increase recurring engagement and trade volume
- Reduce friction for long-term investors

Scale:

- Assume tens of thousands of users with active plans
- Each plan executes once per month, but the execution day is chosen by the user
- This means plan executions happen daily across the platform

The Feature

Users can create an investment plan through the BUX app.

A plan defines a recurring investment that automatically executes and creates orders based on the plan configuration.

Investment Plan (App)
Plan Name: "Monthly Tech Portfolio"
Status: Active
Investments:
— AAPL — €100
— AMZN — €50
— GOOGL — €50
Recurrence: Monthly (on day chosen by user)
Next execution: 1 Feb 2026
Funding: from the user's available cash balance

Requirements

Core:

- User can create plan(s)
- Plan configuration contains:
 - Day of the month when the plan should be executed
 - List of asset IDs and amount to invest per asset
- User can view their plan(s)
- Plan executes once per month on the configured day → creates order(s)
- Plans are funded from the user's available cash balance
- When a plan executes, it creates one order per instrument (asset ID) defined in the plan configuration

Note: The above covers functional requirements. As part of your design, consider non-functional requirements (e.g., scalability, reliability, observability) and document any assumptions you make.

Open Questions & Challenges

As part of your design, address the questions and challenges you find most critical.

Design decisions (you decide):

- How is execution triggered?
- How do you track execution state?

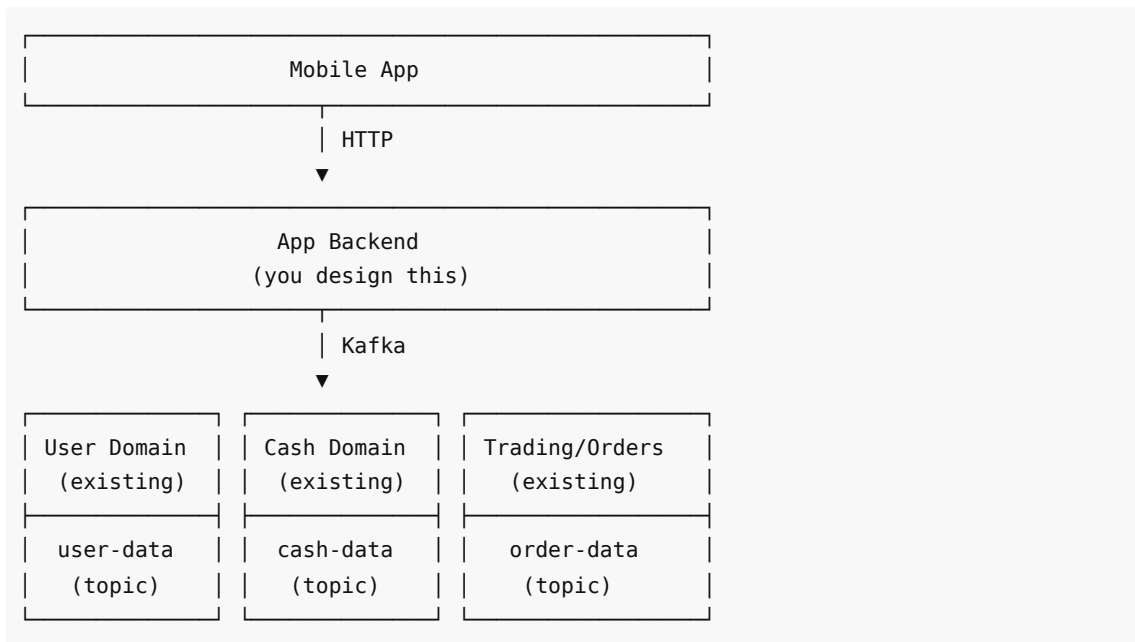
Challenges to consider:

- What happens if the execution of a plan fails halfway through? What happens to the orders that are part of the plan?
- What if the user modifies the plan while an execution is in progress?
- How do you ensure a plan doesn't execute twice?

- What if the trading engine is unavailable during scheduled execution?
- How do you keep the user informed about execution status?
- What happens if the user's cash balance is insufficient at execution time?

You don't need to solve all of these – prioritize a list for MVP and explain your trade-offs. The list can be much longer; you might also find other challenges not mentioned here.

Technical Context



Given:

- **HTTP** – mobile app communicates with backend
- **Kafka** – all backend service-to-service communication is via Kafka
- **CQRS with event sourcing** – services receive commands, validate against the current state, and emit events

Existing integrations (via Kafka):

- Order commands – place orders by publishing to the Trading/Orders domain
- Order state events – published to `order-data` topic (e.g., order filled, order rejected)
- Cash balance events – published to `cash-data` topic
- Cash allocation during order execution is handled by the Cash Domain (existing) – you don't need to design this

You design:

- Investment Plans services
- HTTP API for create/view plans
- Integration with existing domains via Kafka
- Scheduling and execution of plans

You may use these foundations or propose alternatives – justify your choices.

Use of AI Coding Tools

BUX encourages the use of AI tools to accelerate software delivery and experimentation. As part of this assignment:

Required:

- Document how you leveraged AI tools in your approach
- Describe which tools you used and for what purposes (e.g., architecture brainstorming, diagram generation, research, validation)
- Reflect on what worked well and what limitations you encountered

If you chose NOT to use AI tooling:

- Briefly explain your reasoning

Costs & Reimbursement

BUX will reimburse you a maximum of EUR 100 for costs directly associated with completing this assignment (e.g., AI tool subscriptions, token usage).

To claim reimbursement:

- Provide proof of costs (e.g., invoice, token usage screenshot)
- Include a brief summary of what each tool was used for

Deliverables

Submit **one document** containing:

1. **Architecture overview** – components, data flow, diagrams
2. **Key design decisions** – what you chose, what you rejected, why
3. **Trade-offs** – what are you optimising for? What are you sacrificing?
4. **Risks & mitigations** – what could go wrong? How would you handle it?
5. **Assumptions & open questions** – what would you ask Product/Engineering?
6. **Non-functional requirements** – scalability, reliability, observability, etc. (document your assumptions)
7. **MVP / post-MVP** - how are you gonna deliver the project? Which APIs are you building first?
8. **Delivery plan** - top level overview of development/testing/rollout plan

Optional:

- **Code snippets** – pseudocode or real code illustrating key parts of your design (e.g., execution flow, event handling)
- **SLO considerations** – what would influence them? What trade-offs would that introduce?

Submission Format

Length guidance:

- **Architecture overview:** ~1-2 pages (including diagrams)
- **Key design decisions:** ~1 page
- **Trade-offs:** ~0.5-1 page
- **Risks & mitigations:** ~0.5 page
- **Assumptions & open questions:** ~0.5 page
- **Non-functional requirements:** ~0.5 page
- **MVP / post-MVP:** ~0.5 page
- **Delivery plan:** ~0.5 page

- **Total:** 5–8 pages; appendices optional for supporting details (e.g., code snippets, detailed event flows, data models)

Deliverable Options:

- Inline in a PDF or Markdown document (simplest)
- In a GitHub repository with link included
- With sample test code or pseudocode (optional)

Submission

Subject: Staff Backend Engineer Assignment - [Your Name]

Deadline: 7 days from receipt

Estimated Time: 4–6 hours

Questions?

If you have clarifying questions about the assignment:

- Email us – we're happy to help
- Document your assumptions if you don't hear back
- Part of this exercise is handling ambiguity, but genuine blockers should be clarified

Final Thoughts

There is no single "correct" answer.

We're interested in your architectural thinking, how you navigate trade-offs, and how you communicate complex decisions.

Good luck! We look forward to reviewing your solution.